

Day 48: Basic Encapsulation

Day 48 — Basic Encapsulation

1. Day Number

Day 48

2. Topic Name

Basic Encapsulation

Today you will learn how to avoid changing an object's balance directly and instead access/update it through methods.

3. Connection

Previously, you created `deposit()` and `withdraw()` methods.

You may currently have something similar conceptually to:

```
account.balance
```

Today, you will change the balance attribute to:

```
account._balance
```

and introduce a method such as:

```
get_balance()
```

The idea becomes:

```
Before

Object
|
+-- balance
+-- deposit()
+-- withdraw()

Today

Object
|
+-- _balance
|
+-- get_balance()
+-- deposit()
+-- withdraw()
    |
    +---- controlled access to _balance
```

4. Important Topics

Today's main concepts are:

- `_balance`
 - Getter method
 - Controlled update
 - Encapsulation
 - Object methods
 - `self`
 - Protecting object data from accidental direct modification
-

5. Foundational Notes

What is encapsulation?

Encapsulation means keeping data and the methods that work with that data together inside a class.

For example, a bank account contains:

```
Data:
balance

Behavior:
deposit money
withdraw money
check balance
```

Instead of allowing every part of the program to freely modify the balance, we prefer:

```
User
|
| deposit
v
deposit()
|
v
_balance
```

and:

```
User
|
| withdraw
v
withdraw()
|
v
_balance
```

This gives the class more control over its own data.

Why use `_balance`?

In Python, an underscore before an attribute name is a convention:

```
_balance
```

It means:

```
"This attribute is intended for internal use. Prefer using the class's methods instead of
modifying it directly."
```

For example:

```
self._balance
```

The `_` does **not** make the variable completely private.

Python will still technically allow something like:

```
account._balance
```

But programmers understand that they normally should not directly change it.

So prefer:

```
account.deposit(...)
account.withdraw(...)
account.get_balance()
```

rather than directly changing `_balance`.

What is a getter method?

A **getter** is a method used to retrieve an attribute's value.

Conceptually:

```
def get_something(self):
    return self._something
```

For your bank account:

```
get_balance()
    |
    v
return current _balance
```

This gives callers a controlled way to check the balance.

Controlled update

Imagine someone could do this directly:

```
balance = -500000
```

That could create an invalid account state.

Instead, you want updates to happen through:

```
deposit()  
withdraw()
```

Those methods can first check whether an operation is valid.

For example:

```
deposit  
  |  
  v  
Is amount valid?  
  |  
yes  
  |  
  v  
update _balance
```

Similarly:

```
withdraw  
  |  
  v  
Is amount positive?  
  |  
  v  
Is enough balance available?  
  |  
yes  
  |  
  v  
update _balance
```

This is one of the main benefits of encapsulation.

6. Easy Example

Consider a simple Student class.

Instead of storing:

```
self.marks
```

you might store:

```
self._marks
```

Then provide a method:

```
def get_marks(self):  
    return self._marks
```

And another method could control updates:

```
update_marks()  
  |  
  v  
check new marks  
  |  
  v  
update _marks
```

For example, the method could reject marks below 0 or above 100.

The important idea is:

Data	Methods
<code>_marks</code>	<code><----- get_marks()</code>
^	
<code>+-----</code>	<code>update_marks()</code>

The methods control how the data is accessed or changed.

7. Problem Statement

Extend your existing `BankAccount` class.

Requirements

Store the account balance using:

```
_balance
```

instead of:

```
balance
```

Add a method:

```
get_balance()
```

The method should return the current balance.

Your existing:

```
deposit()
```

and:

```
withdraw()
```

methods should now update:

```
_balance
```

instead of balance.

Your class conceptually becomes:

```
BankAccount
|
|— account_holder
|— _balance
|
|— get_balance()
|— deposit()
|— withdraw()
```

Do not directly modify `_balance` from outside the class during normal use.

8. Concepts Used

You will use:

Class

Defines the structure and behavior of a bank account.

```
class BankAccount:
```

Object

An instance of your class.

```
account
```

Constructor

Used to initialize account information.

```
__init__()
```

self

Represents the current object.

For example:

```
self._balance
```

means:

| the `_balance` belonging to this particular account object.

Attribute

Stores information about an object.

```
account holder  
_balance
```

Method

Defines behavior.

```
deposit()  
withdraw()  
get_balance()
```

Encapsulation

Keeps the object's data and operations together while controlling how the data should be updated.

Getter

Provides a method for retrieving a value.

```
get_balance()
```

Validation

Checks whether an operation is allowed before changing `_balance`.

9. Thought Process

Think through the problem in small steps.

Step 1: What data belongs to the account?

You need:

```
account holder  
balance
```

But today's balance attribute should be:

```
_balance
```

Step 2: How should someone check the balance?

Instead of depending on direct attribute access, provide:

```
get_balance()
```

It should give back the current `_balance`.

Step 3: How should money be added?

Use the existing:

```
deposit()
```

It should validate the amount and then modify:

```
self._balance
```

Step 4: How should money be withdrawn?

Use:

```
withdraw()
```

It should validate:

```
amount > 0
```

and:

```
amount <= available balance
```

Then update:

```
self._balance
```

Step 5: How should balance checking work?

After either operation:

```
deposit / withdraw
  |
  v
get_balance()
  |
  v
current _balance
```

10. Pseudocode

```
CREATE BankAccount class

    CREATE constructor with account holder and balance

        STORE account holder

        STORE balance as _balance

    CREATE get_balance method

        RETURN _balance

    CREATE deposit method with amount

        IF amount is positive

            ADD amount to _balance

        OTHERWISE

            handle invalid deposit

    CREATE withdraw method with amount

        IF amount is not positive

            handle invalid withdrawal

        ELSE IF amount is greater than _balance

            handle insufficient balance

        ELSE

            SUBTRACT amount from _balance

CREATE account object

CHECK balance using get_balance()

DEPOSIT some money

CHECK balance again

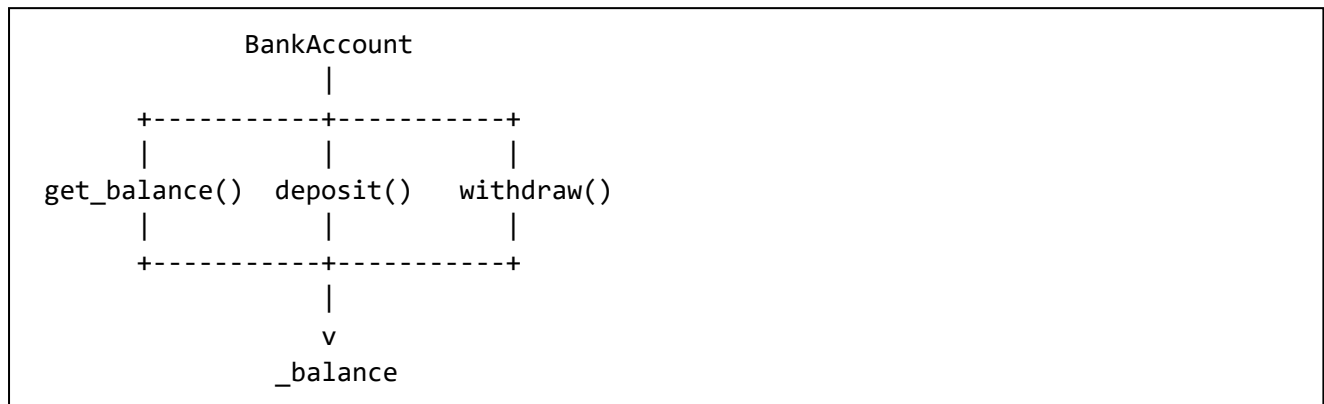
WITHDRAW some money

CHECK balance again
```

11. Suggested Solving Approach

Use an **OOP-based approach**.

Keep all balance-related behavior inside BankAccount:



A useful rule to remember is:

| The object should be responsible for protecting and updating its own data.

So outside code should say:

```
"deposit this amount"
```

rather than:

```
"manually calculate and replace the balance"
```

12. Easy Edge Cases

Edge Case 1: Check balance after deposit

Suppose:

```
Starting balance = 1000
Deposit = 500
```

After the deposit:

```
_balance = 1500
```

Calling `get_balance()` should give:

```
1500
```

Edge Case 2: Check balance after withdrawal

Suppose:

```
Starting balance = 1500
Withdrawal = 300
```

After withdrawal:

```
_balance = 1200
```

Calling `get_balance()` should give:

```
1200
```

Edge Case 3: Invalid deposit

Suppose:

```
Deposit = 0
```

or:

```
Deposit = -100
```

The balance should remain unchanged.

Edge Case 4: Withdrawal greater than balance

Suppose:

```
_balance = 1000
withdrawal = 1500
```

The withdrawal should not happen.

After checking the balance:

```
get_balance() → 1000
```

13. Expected Input

For today's exercise, you can keep the input simple rather than using `input()`.

For example, conceptually create an account with:

```
Account holder: Rahul
Starting balance: 1000
```

Then perform:

```
Check balance

Deposit: 500
```

```
Check balance  
  
Withdraw: 300  
  
Check balance
```

So the sequence is:

```
Create Account  
  |  
  v  
Balance = 1000  
  |  
  v  
Deposit 500  
  |  
  v  
Balance = 1500  
  |  
  v  
Withdraw 300  
  |  
  v  
Balance = 1200
```

14. Expected Output

Your exact wording can differ.

A possible result would look conceptually like:

```
Current balance: 1000  
Current balance: 1500  
Current balance: 1200
```

If you also print messages inside deposit and withdrawal methods, your output could contain additional messages such as successful deposit or withdrawal information.

The important balance progression is:

```
1000  
↓ +500  
1500  
↓ -300  
1200
```

15. Hint Only

Start from your **Day 47 BankAccount class** rather than creating everything again.

Make these three main changes:

```
1. balance
   ↓
   _balance

2. Add:
   get_balance()

3. Make sure:
   deposit() → updates _balance
   withdraw() → updates _balance
```

For the getter, ask yourself:

```
What single value does get_balance() need to return?
```

For the other methods, search your existing code for every use of:

```
self.balance
```

and think about what it should become now.

Do not directly write a new balance from outside the object. Try to perform normal balance changes only through `deposit()` and `withdraw()`.

Small challenge

After creating your object, test this sequence:

```
get_balance()
  ↓
deposit(500)
  ↓
get_balance()
  ↓
withdraw(300)
  ↓
get_balance()
```

If the three balance values change correctly, your basic encapsulation is working.